```
puts square
```

produces:

```
30
some shape
```

To be fair, this syntax is pretty rare in actual Ruby code.

By making square block-local, values assigned inside the block will not affect the value of the variable with the same name in the outer scope.

Ruby also offers a shortcut way to access the arguments to a block based on their numerical position. Where before we wrote our block like this:

```
[1, 2].each { |thing| puts thing }
```

you can instead use the special variable _1 to indicate the first positional argument to the block, meaning you can write this as:

```
[1, 2].each { puts _1 }
```

If the block had more arguments, you could reference them as _2, _3 and so on. We think that if this goes past _1, you are probably better off giving the block variables their own names.

This version is shorter, but can be harder to read if the name of the argument was conveying important information. Later, we'll see another common shortcut for simple block invocations.

Iterators

A method that can invoke a block of code repeatedly for one or more elements is sometimes called an *iterator* or an *enumerator*. We said earlier that a block may appear only in the source adjacent to a method call and that the code in the block is not executed at the time it is encountered. Instead, Ruby remembers the context in which the block appears (the local variables, the current object, and so on—Ruby refers to all of this information as a *binding*) and then enters the method that was called. This is where the magic starts.

Within the method, the block may be invoked, almost as if it were a method itself, using the yield statement. Whenever a yield is executed, it invokes the code in the block passed to the method. If there is no block, Ruby throws an error. When the block exits, control picks back up immediately after the yield. Let's start with a trivial example:

Why "yield"?



Programming-language buffs will be pleased to know that the keyword yield was chosen to echo the yield function in Liskov's language CLU, a language that is more than forty years old and yet contains features that still haven't been widely exploited by the CLU-less.

```
def two_times
  yield
  yield
end
two_times { puts "Hello" }
```